Final Project Report

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	10
Clarity	2					
Content Quality	4					
Spelling & Grammar	2					
Organization and Formatting	2					
					Total obtained mark	
Comments						

Semester: Spring 2026

Student Name: Kawshik Kumar Saha, Md. Fazly Rabby, Md. Munjurul Islam

Student ID: 0242310005341289, 0242310005341077, 0242310005341104

Batch: 40 **Section: E1**

Course Code: SE331

Course Name: Software Engineering Design Capstone Project

Course Teacher Name: Rahat Uddin Azad

Designation: Lecturer, Dept. of SWE, Daffodil International University

Submission Date: 16/04/2026

1. Executive Summary

The process of transitioning to a foreign country's university is often a challenging and overwhelming experience for students. Prospective international students face a significant real world problem: critical information regarding university academic programs, exact tuition fees, language requirements, intake schedules, and scholarship eligibilities are scattered across hundreds of disjointed, poorly organized web portals. This manual and inefficient process causes severe pain points, leading to missed application deadlines, overlooked financial aid, and poor decision-making.

StudyGlobe is proposed as a comprehensive solution to this problem. It is a centralized, AI-powered study-abroad preparation platform designed specifically to streamline the entire university selection and assessment process. The core value proposition of StudyGlobe is consolidating all vital admission data into a single, intuitive interface while providing intelligent tools—such as a personalized profile-based recommendation system and an interactive AI Chatbot—to guide students naturally.

Technology Overview: The project is built on a modern, robust, and scalable full-stack architecture.

Frontend: React 18, React Router v6, and Tailwind CSS for a premium, responsive UI.

Backend: Node.js, Express.js, and Custom JWT Authentication for secure routing.

Database: PostgreSQL hosted via Supabase, utilizing strict Foreign Key relationships and Row Level Security (RLS).

AI Integration: Google Gemma (gemma:latest) via Ollama local inference engine, acting as an intelligent agent.

Team Information-

Team Member: 01 (Team Lead)

Full Name: Kawshik Kumar Saha

Student ID: 0242310005341289

Role: Database design, Documentation preparation, Testing, Backend Design AI Integration Developer.

Team Member: 02 (Frontend Developer)

Full Name: Md. Fazly Rabby

Student ID: 0242310005341077

Role: Frontend development, UI/UX design.

Team Member: 03 (Backend Developer)

Full Name: Md. Munjurul Islam

Student ID: 0242310005341104

Role: Backend development, Data integration

2. Introduction

2.1 Problem Statement

The process of preparing for abroad studies relies heavily on manual and inefficient routines. Students must navigate a sheer volume of unstandardized data—from matching their academic scores against arbitrary university constraints to figuring out language qualifications and strict intake windows. Because there is no central mechanism validating a student's profile against multiple global universities simultaneously, students are often forced to use expensive third-party admission agencies or rely on outdated information.

2.2 Project Objectives

To effectively solve the identified problem, we set out to achieve the following core objectives:

1. **Centralize Data Repository**: Create a structured database incorporating universities, intakes, languages, and scholarships for multiple countries in one place.

2. **Automated Personalization:** Implement a Recommendation System that analyzes a user's profile info and maps it directly to eligible universities and scholarships.

3. **Intelligent Assistance**: Provide real-time, context-aware assistance via an AI Chatbot equipped with function-calling capabilities to query the database using natural language.

4. **Admin Data Management**: Allow authorized personnel (Admins) to intuitively manage and moderate all platform data through a dedicated dashboard without needing SQL knowledge.

2.3 System Boundaries (Scope)

To ensure effective scope management, the operational boundaries of the system have been clearly defined:

In-Scope: 1. Secure user registration/authentication, 2. User profile creation, 3. Browsing and filtering university catalogs, 4. AI chatbot querying, 5. Automated university recommendations, and 6. Admin dashboard for updating records.

Out-of-Scope: 1. Direct student application submissions to external universities, 2. Visa processing/tracking, 3. Flight accommodation arrangements, and 4. Integrated payment gateways for tuition.

2.4 Stakeholder Analysis

Student (Primary End-User): Needs an intuitive, frictionless system to log in, maintain their academic profile, seek accurate information natively via AI, and receive university matches to fast-track their preparation.

System Administrator (Admin): Needs a secure portal to manage users, insert newly partnered universities, update critical intake deadlines, and ensure system integrity.

3. Requirement Specification

3.1 Functional Requirements (FR)

Require-ment ID	Feature Name	Description	Priority	Status
FR-01	Secure Enrollment and Authentication	The system shall allow users to register and log in securely.	High	Implemented
FR-02	User Profile Management	The system shall allow users to create and update their academic details and preferences.	High	Implemented
FR-03	Country Catalog	The system shall display a list of available countries for study abroad.	High	Implemented
FR-04	View Country-wise Universities	The system shall allow users to view universities based on selected country.	High	Implemented
FR-05	University Details	The system shall display detailed information about a selected university.	High	Implemented
FR-06	Academic Program Exploration	The system shall allow users to view programs offered by a selected university.	High	Implemented
FR-07	View Intake Schedule	The system shall display available intake sessions for a university.	Medium	Implemented

FR-08	Language Requirement Inspection	The system shall display language test requirements for programs.	Medium	Implemented
FR-09	Scholarship Eligibility Assessment	The system shall display scholarship eligibility information for universities.	Medium	Implemented
FR-10	Admin Data Management	The system shall allow administrators to manage stored data.	High	Implemented
FR-11	Preference-Based Filtering	The system shall allow users to filter universities based on preferences.	High	Implemented
FR-12	AI Chatbot Widget	A floating AI Assistant to answer natural-language queries by pulling database info contextually.	High	Implemented
FR-13	Profile-based Personalized Recommendations	A system recommending scholarships and universities strictly based on user profile inputs.	High	Implemented

3.2 Non-Functional Requirements (NFR)

Performance & Latency: The system must resolve standard database queries in under 500ms. The Agentic AI Chatbot must process queries, trigger function calls, and return conversational data within 3-4 seconds to ensure interactive fluidity.

Security: Passwords must be strictly hashed (bcrypt). API endpoints modifying or exposing user data must be protected under bearer tokens (JWT), and the React application must employ interceptors to process and safeguard those tokens.

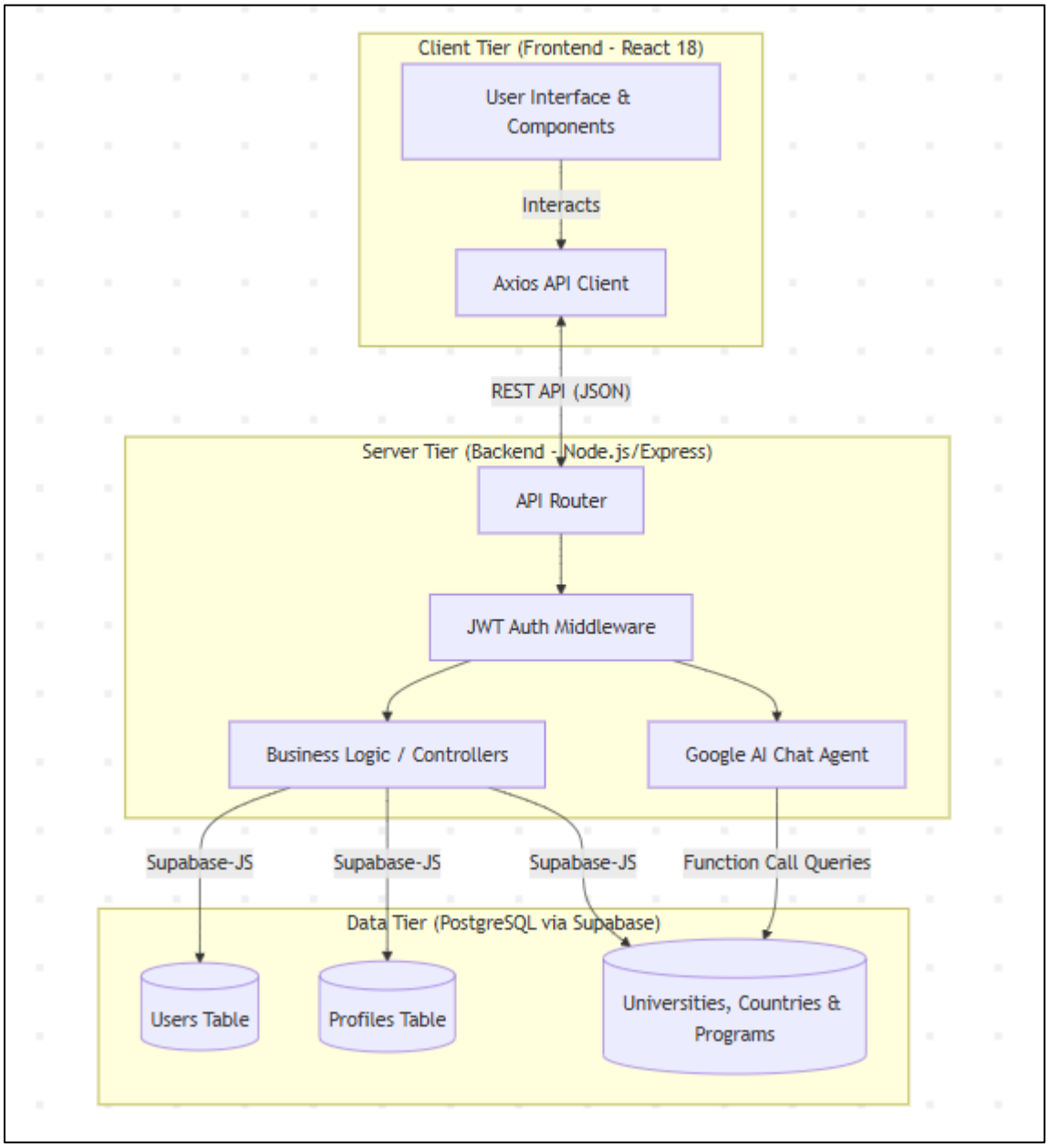
Usability: The interface must strictly observe modern UI/UX principles, utilizing Tailwind CSS for full mobile responsiveness, color contrast legibility, and state-of the-art formatting.

Reliability & Availability: Utilizing PostgreSQL on Supabase provides high availability and fault-tolerant cloud data integrity with real-time scalability.

4. System Design & Architecture

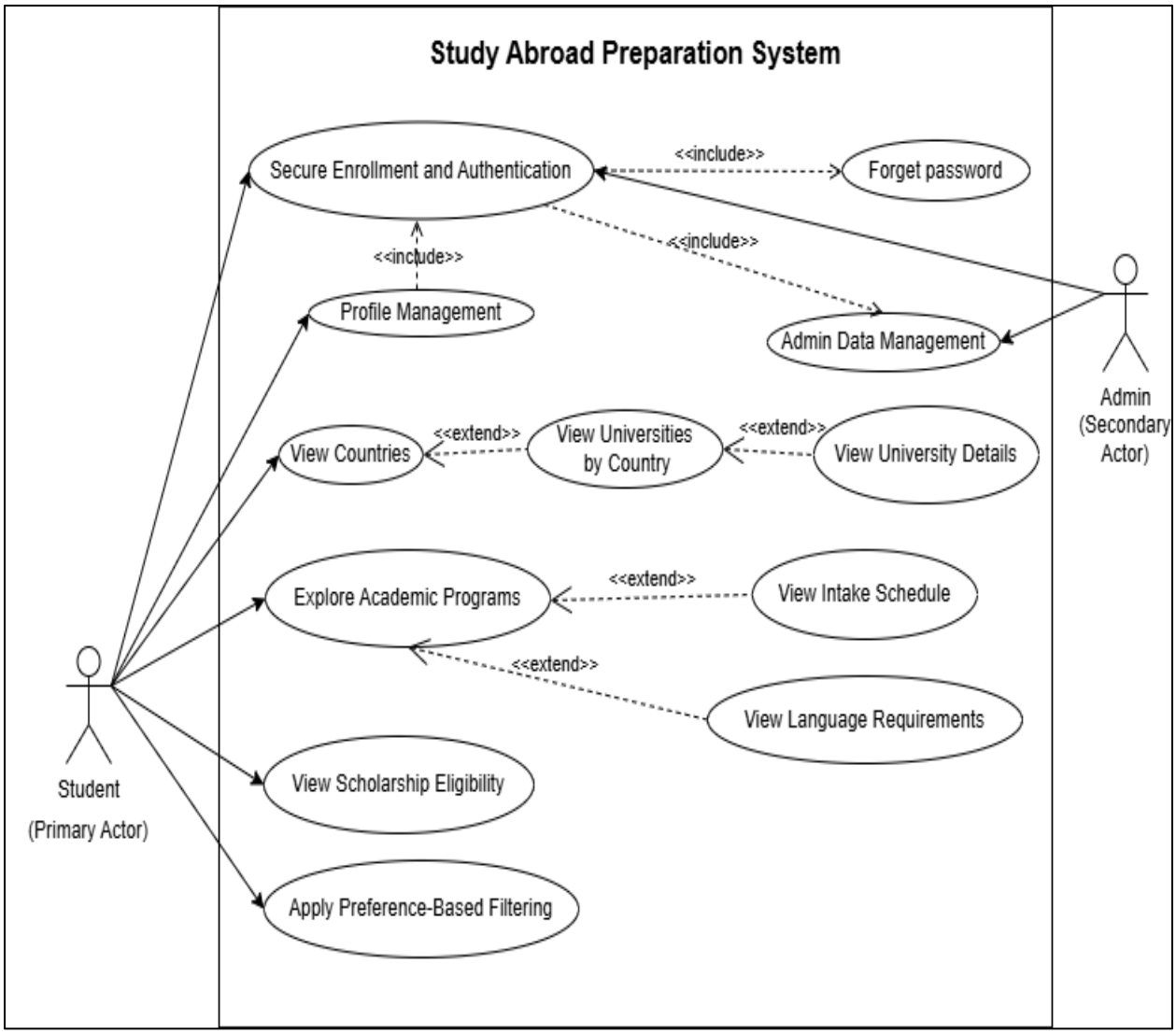
4.1 System Architecture Diagram

A high-level block diagram defining the structural flow of our Client-Server-Database architecture.

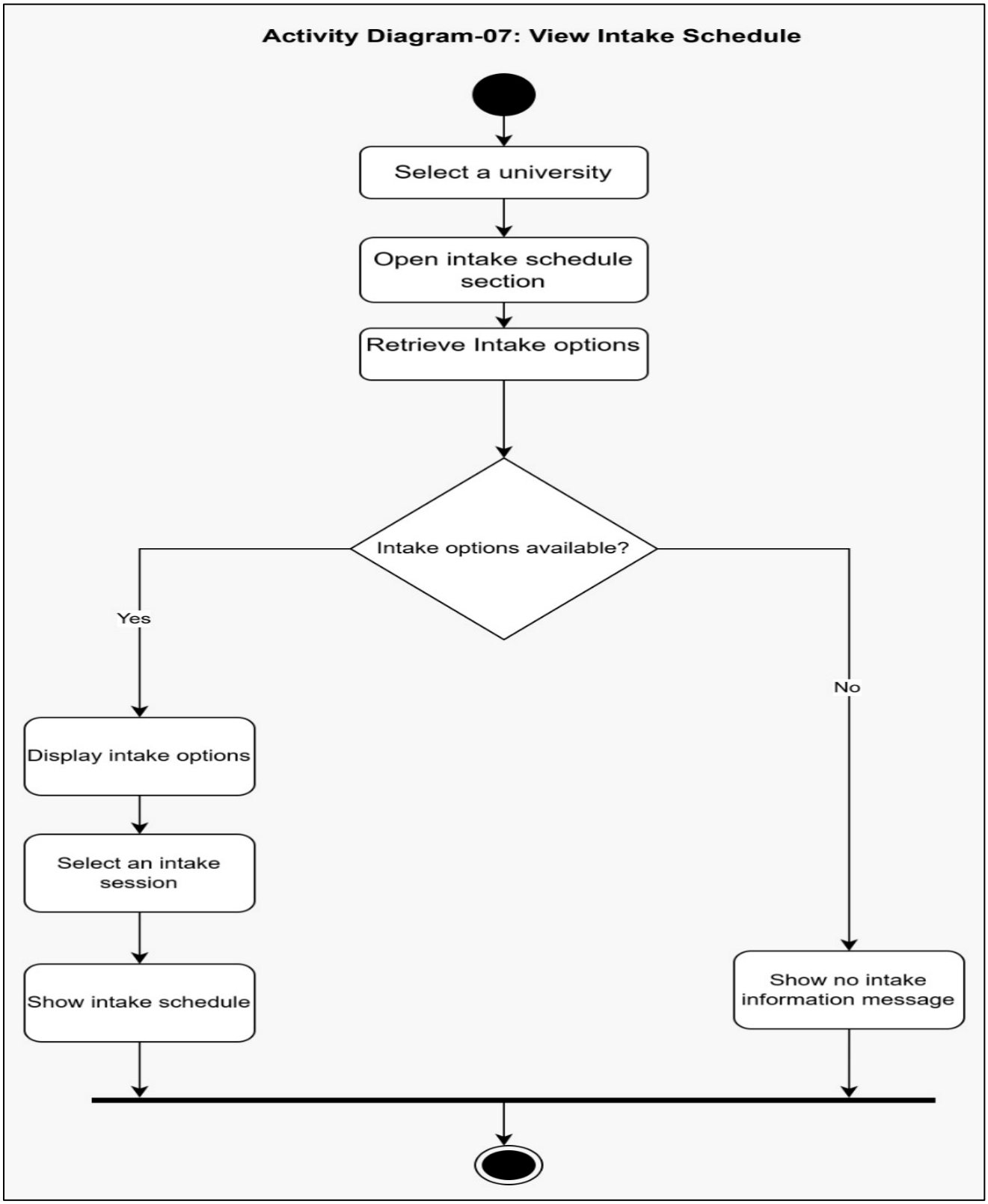


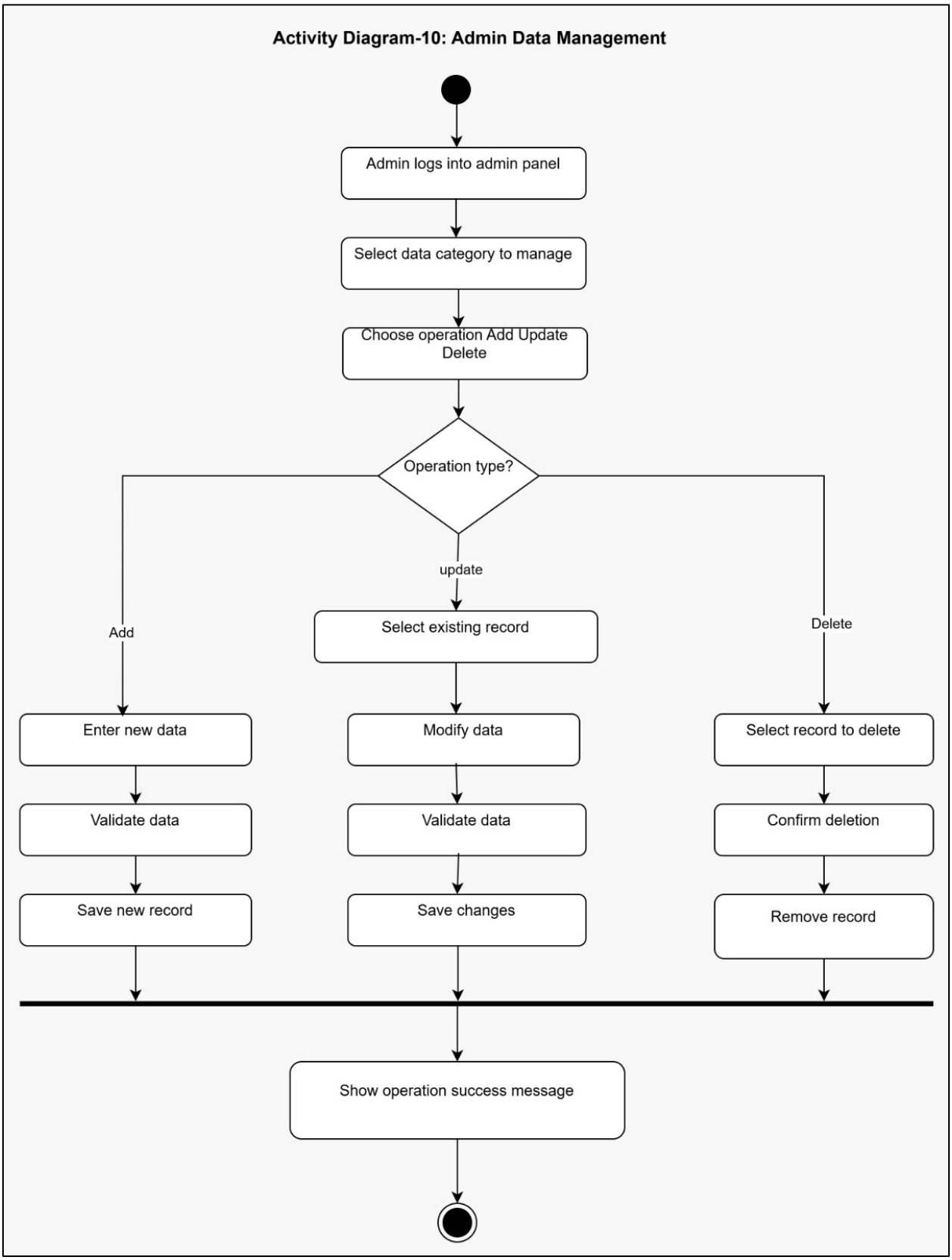
4.2 System Modeling

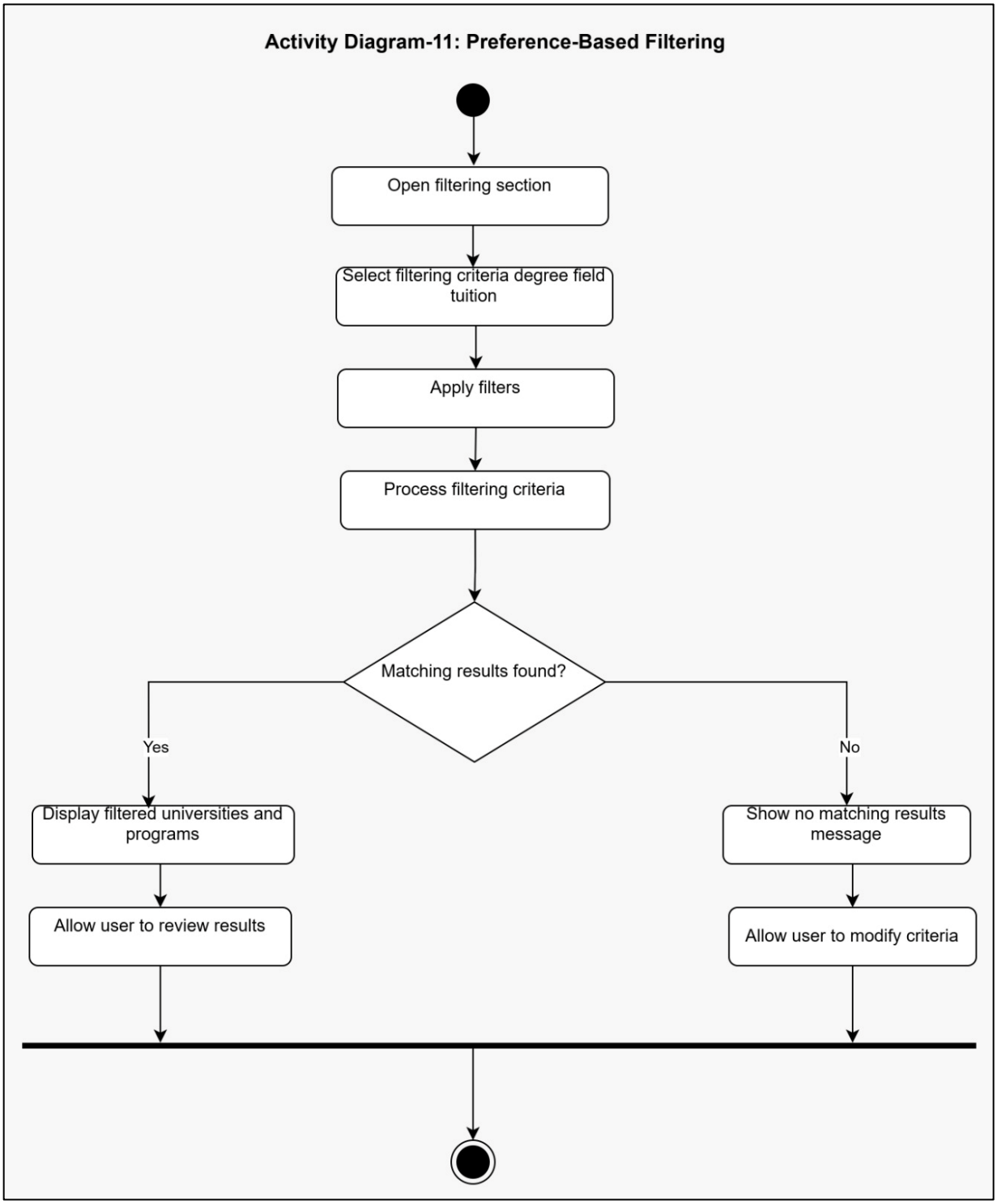
Use Case Diagram:



Activity Diagrams:

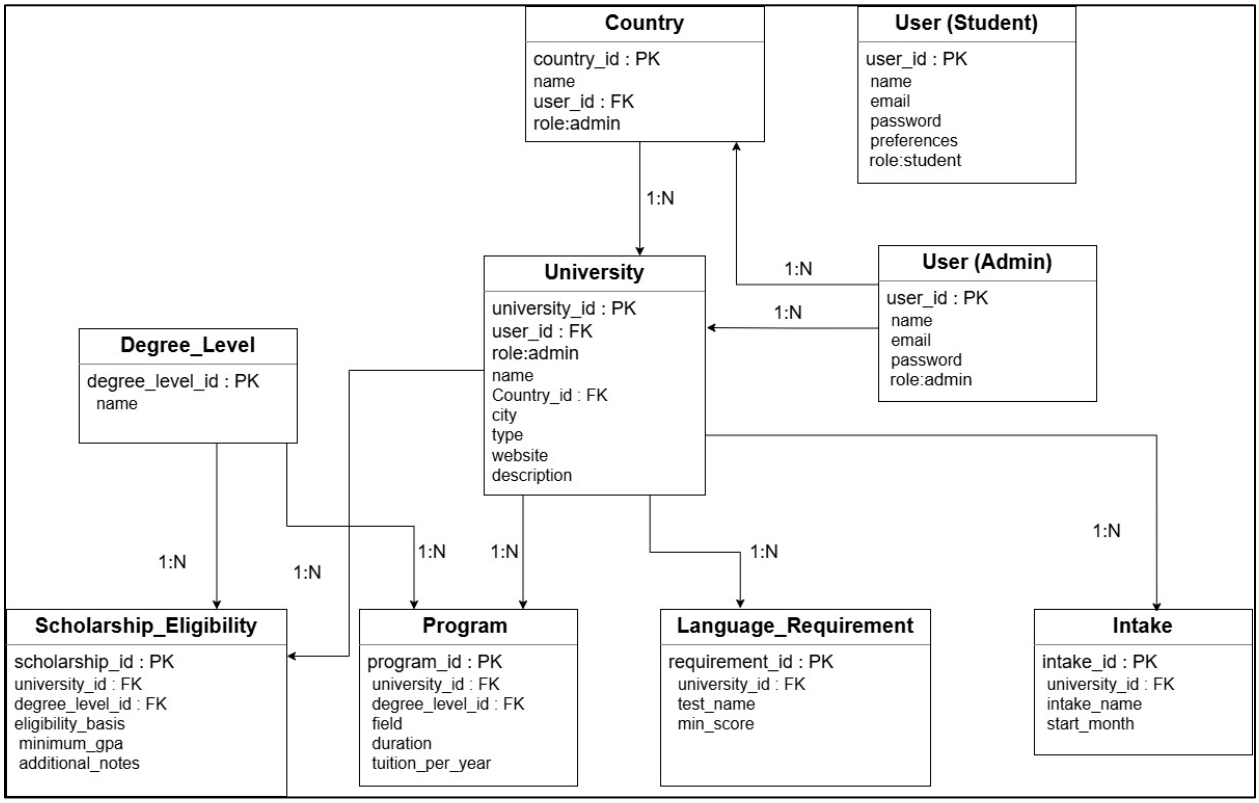



Activity Diagram-10: Admin Data Management
Admin logs into admin panel
Select data category to manage
Choose operation Add Update Delete
Operation type?
Add
update
Delete
Select existing record
Enter new data
Modify data
Select record to delete
Validate data
Validate data
Confirm deletion
Save new record
Save changes
Remove record
Show operation success message


Activity Diagram-11: Preference-Based Filtering
Open filtering section
Select filtering criteria degree field tuition
Apply filters
Process filtering criteria
Matching results found?
Yes
No
Display filtered universities and programs
Show no matching results message
Allow user to review results
Allow user to modify criteria

4.3 Database Design

Entity Relationship Diagram (ERD)



Data Dictionary:

Field Name	Type	Constraints	Description
id (users)	UUID	Primary Key(PK)	Unique user identifier inside the system
email (users)	VARCHAR	Unique, Not Null	User login email address
password_hash (users)	VARCHAR	Not Null	Hashed credential string for security
is_admin (users)	BOOLEAN	Default (false)	Sets Admin privileges
user_id (profiles)	UUID	PK, Foreign Key	Links exactly 1:1 to users table

country_id (universities)	INT	Foreign Key	Links an institution to a country
University_id(programs)	INT	Foreign Key	Links programs, intakes, language requirements, scholarship to an university

5. Technical Implementation

5.1 Backend / Data Interaction Documentation:

Method	Endpoint URL	Action
POST	/api/auth/register	Inserts new user into the database and hashes passwords securely.
POST	/api/auth/login	Validates credentials and replies with a secure JWT token.
GET	/api/profile	Fetches the user profile linked to the extracted JWT token.
PUT	/api/profile	Updates the student's background metrics (GPA, Degree, Budget).
GET	/api/universities?country=...	Retrieves all universities dynamically matching provided filters.
POST	/api/chat	Protected route that forwards natural language payloads.

5.2 Role-Based Access Control (RBAC)

Role distinction happens systematically on the Express server level utilizing specialized Middleware (auth.js). Every time a user logs in, the users table outputs the is_admin boolean into the signed JWT token.

1. General User: Reaches standard protected routes via the authenticateUser middleware that only checks token signature validities.

2. Admin: Endpoints assigned for modification (e.g., adding Universities) pass through a distinct requireAdmin middleware. If req.user.is_admin is false, it returns a 403 Forbidden response, preventing any unauthorized queries from being executed.

To clarify RBAC, these tables are needed- users, roles, permissions, role_permissions.

roles:

role_id(PK)	**role_name**
1	student
2	admin

users:

user(Student)
user_id (PK) name email password role_id (FK)

user(Admin)
user_id (PK) name email password role_id (FK)

permissions:

permission_id (PK)	**permission_name**
1	view_data
2	manage_data
3	manage_users
4	filter_data
5	search_university
6	check_scholarship_eligibility

role_permissions:

role_permission_id (PK)	**permission_id**	**role_id (FK)**
1	1	1

2	2	2
3	3	2
4	4	1
5	5	1
6	6	1

5.3 Frontend Organization

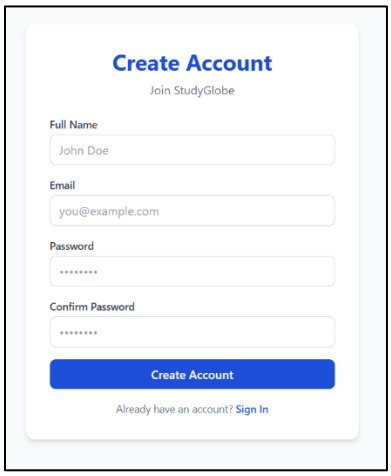
The architecture follows Atomic Design Principles to enforce code re-usability and cleaner DOM structuring:

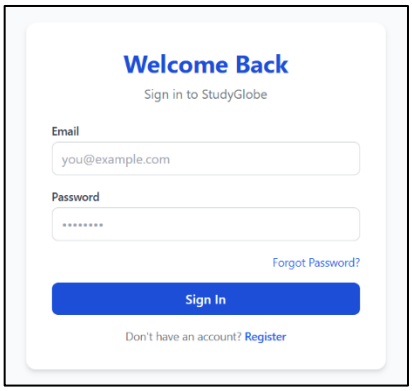
- **/components:** Holds reusable and stateless UI shards (e.g. Navbar, Chatbot, ProtectedRoute).
- **/pages:** Holds holistic dashboard views containing larger workflows (e.g. AdminDashboard, Universities).
- **/context:** Holds global contexts like ContextAPI to prevent "prop drilling" (e.g., AuthContext.js persists login state).
- **/services:** Holds dynamic Axios instances with interceptors (api.js) configured uniformly.

6. User Manual

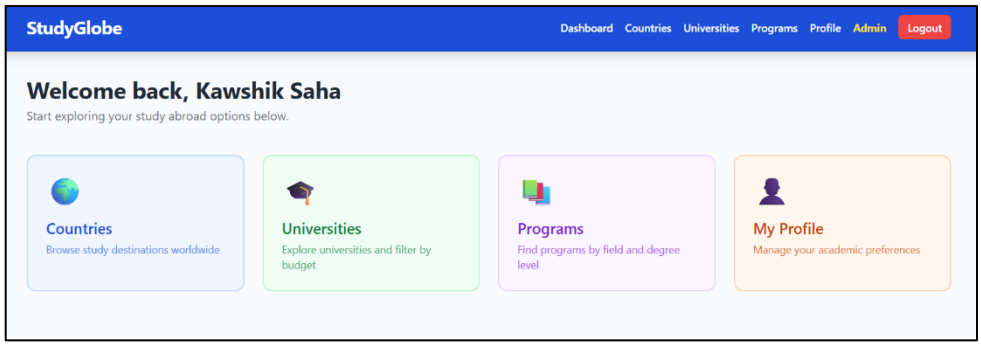
6.1 Getting Started

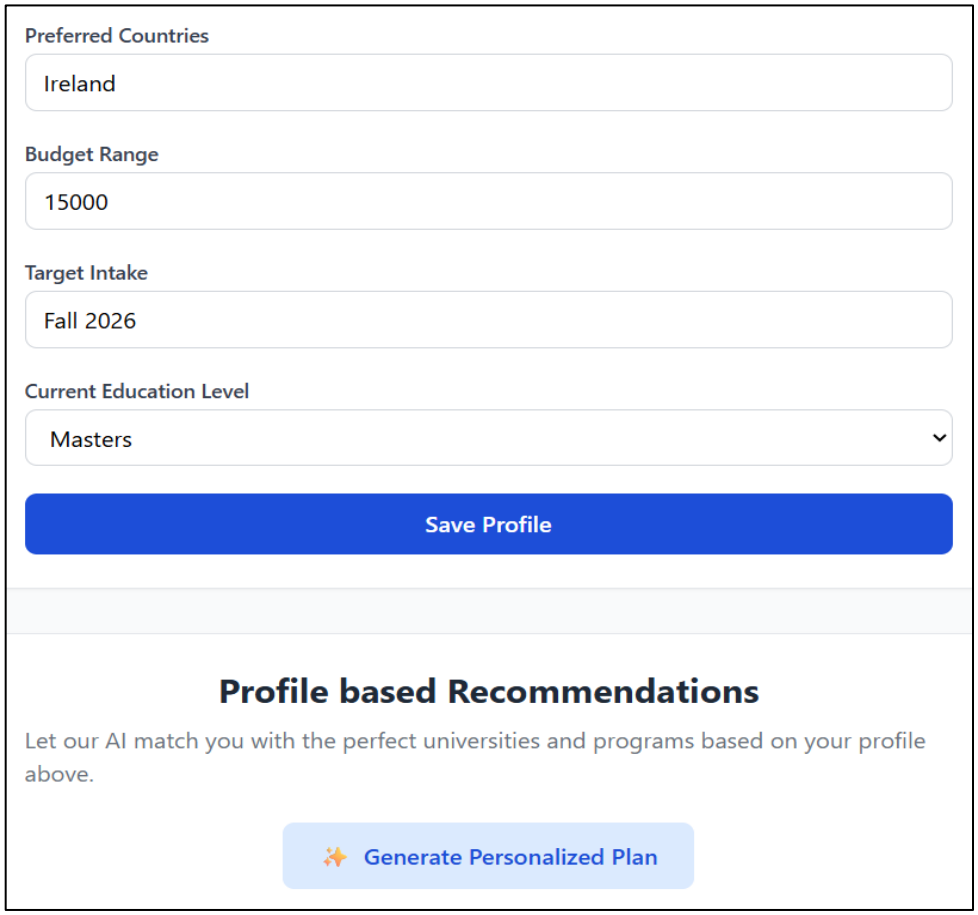
Step 1: Access the application homepage and click the Register button.





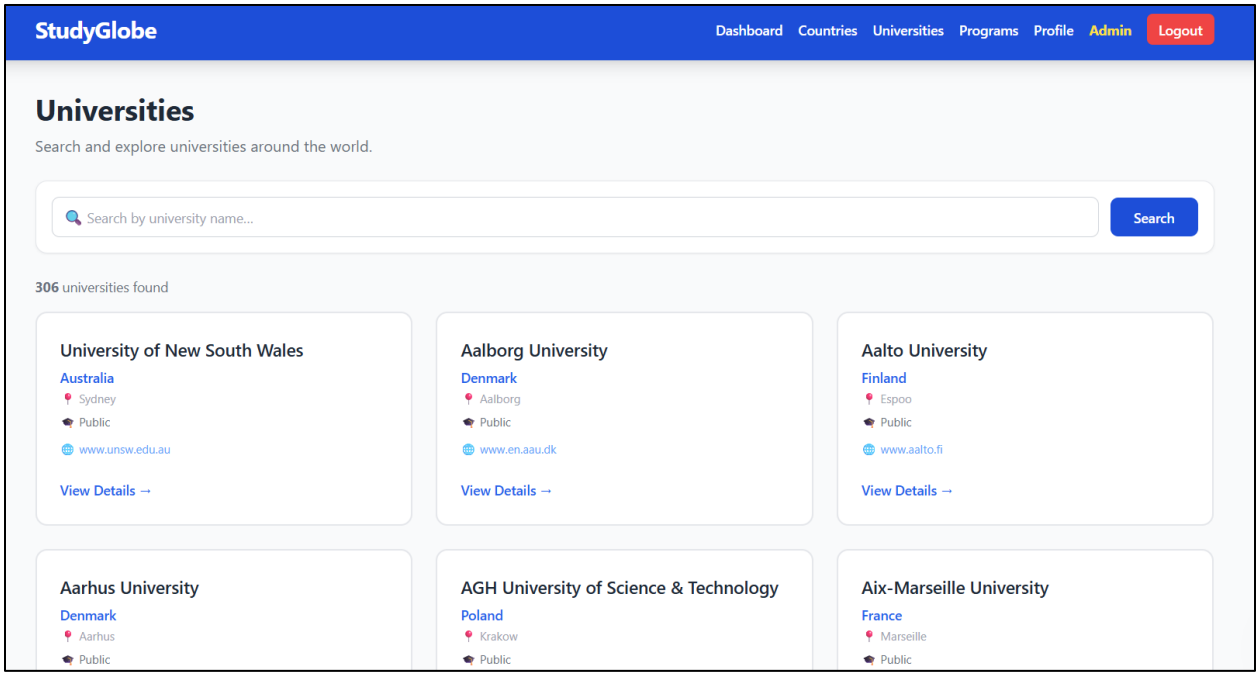
Step 2: Fill in your desired Email and secure Password, then log in. The application will direct you to a Setup window where you will build your academic profile (adding Name, GPA, etc.).





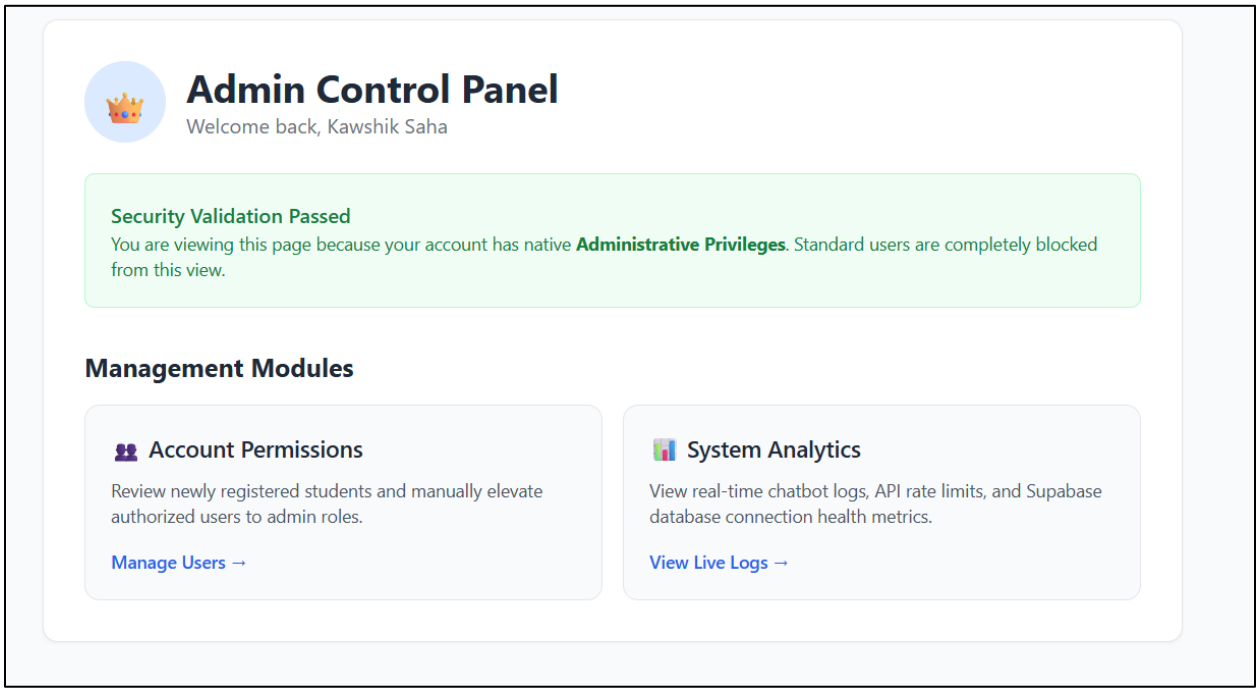
6.2 Feature Walkthroughs

FR-4: Browsing Countries & Universities Catalog



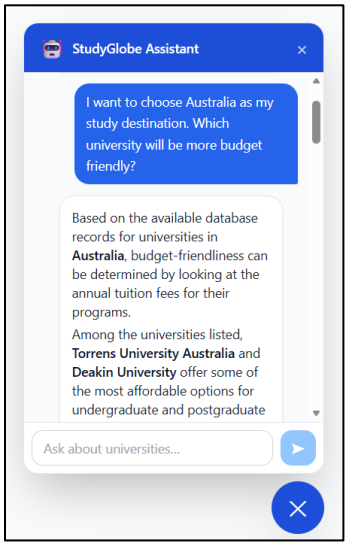
Navigate to the "Universities" tab. Filter by specific countries using the clean Tailwind UI cards. Click any University Card to dynamically fetch its Programs, Intakes, and Language constraints.

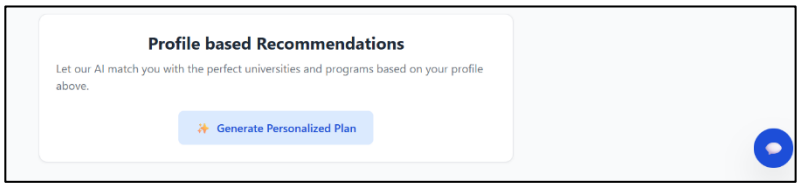
FR-10: Utilizing the Admin Dashboard (Admins Only)



If your account holds Admin clearance, click the "Admin Panel" tab appearing on the navbar. Here, you can click "Add New Recording" and instantly update or insert brand new academic programs or countries via the GUI, eliminating the need to write Raw SQL.

FR-12: Utilizing The AI Chatbot Assistant





Look for the blue floating beacon on the bottom-right of the dashboard. Click it to unfurl the Chat Window. Type queries naturally like "What universities in Japan offer computer engineering scholarships?". Hit Enter, and the agentic backend will formulate a specialized answer derived from our customized Postgres tables.

7. Quality Assurance & Testing

7.1 Test Case Scenarios

Test ID	Test Description	Input Data	Expected Result	Actual Result	Pass/Fail
TC-01	User Registration & Login	Valid Email, Password	User is registered, JWT token securely assigned.	HTTP 201 created. JWT decoded properly.	PASS
TC-02	Secure Route Protection	Trying to access /api/profile without JWT setup	HTTP 401 Unauthorized Response.	Access denied correctly.	PASS
TC-03	AI Chatbot Data Retrieval	Query: "Find colleges in Canada"	Chatbot queries universities database and outputs results.	Renders markdown list correctly.	PASS
TC-04	Admin Access Enforcement	Non-admin tries to POST a new University	HTTP 403 Forbidden Response.	Prevents writing data accurately.	PASS

TC-05	Profile Metrics Update	Add Name=Fahim, GPA=3.5	Updates the user_profiles table bound by foreign key constraints.	Profile reflects immediately in Dashboard.	PASS

7.2 Requirements Traceability Matrix (RTM)

Requirement ID	Feature Name	Mapped Test Cases
FR-01	Secure Authentication	TC-01, TC-02
FR-02	Profile Management	TC-05
FR-10	Admin Control Panel	TC-04
FR-12	AI Chatbot Widget	TC-03

8. Individual Contribution

Kawshik Kumar Saha

Technical Contribution: Engineered the core backend infrastructure utilizing Node.js/Express. Implemented the strict Custom JWT logic (auth.js middleware) and seamlessly integrated the Agentic AI Chatbot using Google Gemini's function-calling mechanism. Mentored the data ingestion pipeline.

Technical Challenges: Transitioning from Supabase's native auth to a fully custom JWT framework presented severe payload integrity challenges. Furthermore, managing the asynchronous function-call logic for Gemini while handling potential timeout scopes proved deeply complex. Resolved it by building a structured iterative loop mapping queries directly to PostgreSQL.

Learning Outcomes: Deepened understanding of server-side security architectures, the orchestration of large language models via backend pipelines, and the implementation of robust Row Level Security rules in Supabase.

Md Fazly Rabby

Technical Contribution: Served as the Frontend Developer and UI-UX lead. Built the vibrant, responsive dashboards utilizing React 18 and Tailwind CSS. Implemented complex state management for Chatbot components and constructed interceptors within Axios (api.js) to systematically handle session tokens without fail.

Technical Challenges: Structuring the floating AI Widget to parse Markdown seamlessly without over-expanding its DOM boundaries was structurally tedious. Handled complex prop states across the Admin Dashboard for dynamic table updates without triggering full page reloads.

Learning Outcomes: Advanced efficiency in atomic React hooks structuring. Proficiently mastered Tailwind Typography constraints and refined skills in executing HTTP client lifecycle manipulations (Axios Interceptors).

Md Munjurul Islam
Technical Contribution: Acted as Database Administrator and Quality Control tester. Normalized the PostgreSQL database up to 3NF logic, resolving critical Foreign Key constraint violations linking users with user_profiles. Validated standard system processes across all NFR metrics.
Technical Challenges: Handling disjointed data constraints when linking Universities directly to Intakes, Scholarships, and Language Requirements tables. Addressed multiple CASCADE DELETE failures which required refactoring how primary keys were indexed via Supabase scripts.
Learning Outcomes: Gained robust proficiency in Relational Database Management Systems (RDBMS). Conceived deep appreciation for software testing strategies and requirement matrix validations across multi-tier setups.

9. Conclusion & Future Work

9.1 Conclusion

The StudyGlobe project successfully engineered a scalable and intelligent alternative to manual study-abroad preparation. By rigorously isolating responsibilities across a robust React Frontend, a secured Express REST API, and a deeply optimized PostgreSQL database, the system effectively centralizes data. The inclusion of Artificial Intelligence pushes the traditional directory boundaries further, actively serving as an autonomous mentor. The project proves that leveraging unified, properly structured tech stacks natively solves critical inefficiencies experienced by today's international student candidates

9.2 Future Enhancements

In Version 2.0, we propose implementing:
1. Visa Processing Tracking: A dynamic Kanban board visually mapping a user's visa application stages automatically validated by AI.
2. Peer-to-Peer Mentorship Chat: Implementing WebSockets (Socket.IO) to allow new students to chat with authorized alumni natively.
3. Global Payments Gateway: Native integration via Stripe to directly allow tuition down-payments through the platform.

10. Appendices

Appendix A: GitHub Repository

https://github.com/learner1289/Study-Abroad-Preparation-System

Appendix B: Installation & Setup Guide

Step 1: Install Dependencies Open terminals in both project directories:
In your backend folder:

```
cd backend
npm install
# In your frontend folder:
cd frontend
npm install
```

Step 2: Setup Environment Variables Create .env inside backend/ and frontend/:

```
PORT=5000
SUPABASE_URL=https://fgwkzweywzmpnebnrwvm.supabase.co
SUPABASE_SERVICE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBh
YmFzZSIsInJlZiI6ImZnd2t6d2V5d3ptcG5lYm5yd3ZtIiwicm9sZSI6InNlcnZpY2Vfcm9sZSIsIm
lhdCI6MTc3MDUzNzUxMywiZXhwIjoyMDg2MTEzNTEzfQ.VNhrnk_Qm20FzC0v6HlzlsJv
MBUC12vWRkcJwgTWha0
FRONTEND_URL=http://localhost:3000
JWT_SECRET=studyglobe_super_secret_jwt_key_2026
OLLAMA_API_KEY=d2b574828efd410f9e112a9e196e4db4.oR8YUhUhDBY-zEVJWQErqpi0
OLLAMA_HOST=https://api.ollama.com
```

Step 3: Run the Application Start both servers strictly in developer mode:

```
 # Terminal 1 (Backend)
npm run dev
 # Terminal 2 (Frontend)
npm start
```

The interface will deploy locally at http://localhost:3000